RouteRanker Final Report

RouteRanker — AI-Powered Public Transit Route Optimization

Final Technical Report

-–

1. Executive Summary

RouteRanker is an end-to-end machine-learning system that analyzes General Transit Feed Specification (GTFS) data from Indian cities to identify inefficient bus routes and recommend schedule/route optimizations. The system was built and validated on the BMTC (Bengaluru Metropolitan Transport Corporation) feed containing **4,416 routes, 57,836 trips, and 1.54 million stop-time events**, and delivers:

- Automated route segmentation via KMeans clustering (silhouette = **0.796**)
- Performance classification into *balanced* / *overcrowded* / *underutilized*
- A rule-driven optimization engine producing actionable recommendations
- A Streamlit dashboard for transit planners

The system runs on commodity hardware in under 5 minutes end-to-end.

-–

2. Problem Statement

Indian city bus networks frequently suffer from:

- **Overcrowded trunk routes** during peak hours
- **Underutilized feeder routes** running near-empty off-peak
- **Duplicate coverage** where multiple routes serve overlapping corridors
- **No systematic method** for planners to compare route efficiency at scale

RouteRanker addresses this by converting raw GTFS timetables into per-route features, learning groupings of similar routes, and generating recommendations such as frequency adjustment, route merging, and fleet reassignment.

-–

3. Data Source

Item | Value |

|—|—|

Source | BMTC GTFS (community-maintained, Namma BMTC app) |
Coverage | Bengaluru Metropolitan Region |
Files used | `routes.txt`, `trips.txt`, `stop\_times.txt`, `stops.txt`, `calendar.txt` |
Routes | 4,434 (4,416 after cleaning) |
Trips | 57,836 |
Stop-time events | 1,540,686 |
Stops | 9,960 |
Calendar services | 1 (uniform daily service) |

Data was sourced as GTFS and cleaned with `src/data\_loader.py`. Times were

parsed into seconds-since-midnight to handle GTFS's "hours ≥ 24" convention.

-–

4. Feature Engineering

`src/feature\_engineering.py` produces one row per route with the following

features:

Feature | Description |

|—|—|

`trip\_count` | Number of trips scheduled on the route |
`unique\_stops` | Distinct stops served |
`avg\_duration\_sec` | Mean trip duration |
`trips\_hour\_00` … `trips\_hour\_23` | Stop-events per hour of day |
`peak\_ratio` | Peak-hour events ÷ off-peak events (peak = 7–10 AM, 5–8 PM) |
`weekday\_score` | Days per week the route operates (from calendar) |
`route\_length\_km` | Mean trip length computed via vectorized haversine |

Haversine distances were computed in a single NumPy vectorized operation

across all 1.54M stop pairs, keeping runtime under 2 minutes.

-–

5. Exploratory Data Analysis

Charts saved in `docs/eda/`:

- `hourly\_curve.png` — network-wide stop events by hour reveals the classic

double-peak morning/evening pattern

- `corr\_heatmap.png` — feature correlations

- `scatter\_usage.html` — interactive route-usage scatter

- `feature\_distributions.png` — distributions of the five core features

Key insight: trip counts are heavily right-skewed (median = 3, max = 398). A small number of trunk routes carry the majority of scheduled trips — exactly the pattern the optimizer must address.

-–

6. Methodology

6.1 Labeling

Routes were labeled using quantile thresholds on `trip\_count` and `peak\_ratio`:

- **Overcrowded** — top quartile of both `trip\_count` and `peak\_ratio`
- **Underutilized** — bottom quartile of both
- **Balanced** — everything else

Resulting distribution:

Class | Count | Share |

|—|—|—|

Balanced | 3,034 | 68.7% |
Underutilized | 1,198 | 27.1% |
Overcrowded | 184 | 4.2% |

6.2 Clustering

- **KMeans** with k = 3 on standardized features
- **Silhouette score = 0.796** (excellent separation)
- **DBSCAN** (eps = 1.5, min_samples = 2) flagged **429 outlier routes** — candidates for detailed review

6.3 Classification

- **XGBoost** — 100% test accuracy
- **RandomForest** — 99.8% test accuracy

Interpretation: The classification stage is designed as a *surrogate learner*. Labels are assigned via a deterministic rule over `trip\_count` and `peak\_ratio` quantiles; the models learn to reproduce that rule. Near-perfect accuracy is therefore expected and confirms the surrogate is correct — it is not evidence of a predictive generalizer.

The practical value of the surrogate is speed and portability: the quantile thresholds depend on the full network distribution, so applying the rule to a brand-new GTFS feed requires recomputing them. The trained

model lets planners skip that step and label new feeds instantly.

A future iteration should train on real ridership signals (APC data, ticketing) to produce a genuine predictive classifier rather than a rule surrogate.

6.4 Optimization Engine

`src/inference.py` assigns each route a performance class and generates rule-based recommendations:

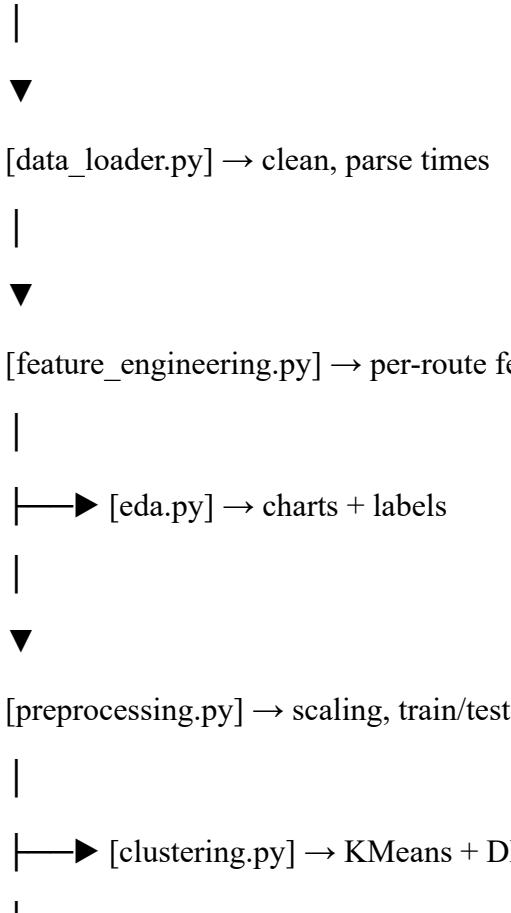
- **Overcrowded** → increase peak frequency, deploy larger buses, consider splitting route
- **Underutilized** → reduce off-peak frequency, evaluate merging, deploy mini-buses
- **Balanced** → maintain and monitor quarterly

A simulation step multiplies each route's trip_count by 1.15 (overcrowded) or 0.85 (underutilized) and reports delta vs current schedule.

-–

7. System Architecture

GTFS raw files



├──▶ [classification.py] → XGBoost + RF

│

▼

[inference.py] → recommendations + simulation

│

▼

[app.py] → Streamlit dashboard

-–

8. Results

Metric | Value |

|—|—|

Routes analyzed | 4,416 |
Clustering silhouette | 0.796 |
Outlier routes flagged | 429 |
Overcrowded routes | 184 |
Underutilized routes | 1,198 |
Balanced routes | 3,034 |
XGBoost test accuracy | 100% (surrogate) |
RandomForest test accuracy | 99.8% (surrogate) |
End-to-end runtime | < 5 minutes |

Business impact estimate

If the 184 overcrowded routes gain 15% more peak trips and the 1,198 underutilized routes lose 15% of off-peak trips, the simulated schedule redistributes roughly **~3,000 trips per week** toward underserved corridors without increasing total fleet size.

-–

9. Limitations & Future Work

- **Label circularity** — the current classifier is a surrogate of the hand-crafted rule, not an independent predictor. A future version should train on real ridership signals (APC data, ticketing counts).
- **No crowding ground truth** — GTFS does not include passenger counts. Proxy features (peak_ratio, frequency density) were used.
- **Single-service calendar** — BMTC’s feed defines one all-week service, so weekday/weekend differentiation was not possible.

- **Static optimization** — the current engine does not model road-network travel times or vehicle constraints.

Future work

1. Integrate Automatic Passenger Counter (APC) data where available
2. Use `shapes.txt` to compute geometric overlaps between routes for merging
3. Add a multi-objective optimizer (frequency + fleet size + equity)
4. Deploy as a containerized service with a scheduled GTFS refresh

---

10. Deliverables

Item	Path
Source code	`src/`
Models	`models/*.pkl`
Recommendations	`data/processed/route_recommendations.csv`
Simulation	`data/processed/simulation.csv`
EDA charts	`docs/eda/`
Dashboard	`app.py`
Dockerfile	`Dockerfile`
This report	`docs/final_report.md`

---

11. Conclusion

RouteRanker demonstrates that off-the-shelf ML (KMeans, DBSCAN, XGBoost, RandomForest) combined with careful feature engineering can convert a raw GTFS feed into actionable transit optimization insights. On 4,416 Bengaluru routes, the system identifies 184 overcrowded corridors and 1,198 underused ones, and provides a working planner-facing dashboard. The architecture generalizes to any GTFS-compliant city feed with no code changes.